

Report Challenge: Learning to Drive with Imitation Learning

Student Number: K51843919

Student Name: Lukas Steu

Setup

I ran the notebook mainly in Google Colab with a Python 3 runtime and used a GPU when available. The task used the CarRacing-v3 environment from Gymnasium with discrete actions. The observations were cropped, resized to 84x84 pixels and converted to grayscale. Therefore, the model input was one grayscale image with values between 0 and 1. The output consisted of 5 logits, one for each possible action.

The required packages, the expert model, and the train/validation datasets were loaded in the setup cells. At the end, the trained model was exported as model.onnx, since this was required for the submission.

Changes of the Code

First, I implemented a CNN as the policy network. I used convolutional layers to extract useful image features and fully connected layers to predict the action. The network outputs logits and not probabilities, because both CrossEntropyLoss and the evaluation script work directly with logits.

I scaled the final logits by 3.0, because the evaluation script samples actions. This makes the policy less random and should reduce bad sampled actions.

For behavioral cloning, I treated the task as a normal classification problem. The image was the input and the expert action was the label. I used CrossEntropyLoss, the Adam optimizer, a learning rate of $3e-4$ and trained for 25 epochs. I logged loss, validation accuracy and entropy, and saved the best model based on validation loss.

Then I implemented DAgger. My policy collected states by driving in the environment, and these states were labelled by the expert. I collected full episodes, because this gives more useful data than only single states. The new expert-labelled states were added to the training dataset and the model was trained again. I collected several rollout episodes per DAgger iteration instead of only one. I used 5 episodes per iteration. I also changed the way the best DAgger model was selected. Instead of only saving the model with the best validation loss, I evaluated the model after each DAgger iteration in the real environment. The model with the best mean driving score was saved, because this is closer to the actual challenge evaluation. I reduced the number of DAgger iterations to 5, since later iterations did not always improve the real driving score.

Results

The behavioral cloning model already learned the basic driving behavior, but it was still not very stable. Sometimes it drove quite well, but in other episodes it left the track early. After using DAgger, the results became better, because the model got more examples from states that it reached by itself.

During DAgger training, I got some good rollout scores, for example 903.05, 907.5, 908.03 and 864.16. This showed that the model was able to drive well, although it was still not perfect in every episode.

In one local evaluation over 50 episodes, I got the following result:

Mean Score: 675.828

Std.: 202.71

Min Score: 212.90

Max Score: 919.90

The score was already close to the 700 threshold, but some weaker episodes lowered the average. The high standard deviation also shows that the policy was still a bit unstable.

Runtime

The setup took about 15s. Behavioral cloning took around 32min. DAgger took longer because new episodes had to be collected and the model was trained again several times - 35min. The final evaluation over 50 episodes took about 10min.

The final model was exported as model.onnx and submitted together with the code and this report.